Experiment 10: Working with Git Branches and Resolving Merge Conflicts Using GitHub

1. Aim

To demonstrate branch management, collaboration, and merge conflict resolution using Git and GitHub.

2. Tools Used

- Git
- GitHub
- Visual Studio Code (VS Code)
- Command Prompt / VS Code Terminal

3. Procedure

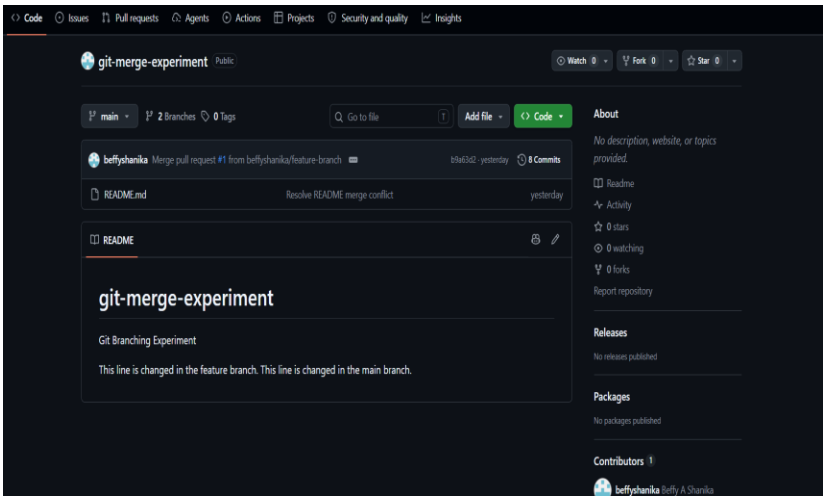
1. Clone the shared repository to the local machine using git clone.
2. Create and switch to a new feature branch using git checkout -b feature-branch.
3. Modify a file such as README.md.
4. Stage and commit the changes using git add and git commit.
5. Push the feature branch to GitHub using git push.
6. Pull the latest changes from the main branch using git pull origin main.
7. Switch back to the feature branch using git checkout feature-branch.
8. Merge the main branch into the feature branch using git merge main.
9. If conflicts occur, open the conflicting files and manually edit the conflicting sections. Save the files after resolving the conflicts.
10. Mark the conflicts as resolved using git add.
11. Commit the merge using git commit.
12. Push the updated feature branch to GitHub and create a pull request.

4. Algorithm

1. Start.
2. Clone the shared repository.
3. Create a feature branch.
4. Modify and commit the files.
5. Push the feature branch to GitHub.
6. Fetch the latest changes from the main branch.
7. Merge the main branch into the feature branch.

8. Check for merge conflicts.
9. If conflicts exist, resolve them manually and stage the corrected files.
10. Commit the merge.
11. Push the updated feature branch.
12. Create a pull request.
13. Stop.

5. Output



6. Result

Git branching and merge conflict resolution procedures were demonstrated. The experiment explained how to integrate changes from different branches and prepare them for collaborative review using GitHub.